

SENTINEL-X

Systeme de surveillance IoT intelligent

Documentation technique complete

Equipe 6 - EPSI (Workshop)

Auteur : rami yahyaoui

Date : 09/10/2026

Sommaire

1. Vue d'ensemble & architecture
2. Matériel ESP32 & capteurs
3. MQTT & sécurité de la communication (TLS, HMAC)
4. Intelligence artificielle prédictive
5. Vision par ordinateur (détection de personnes)
6. État consolidé, affichage & alertes
7. API REST
8. Assistant de calibration (adaptation à l'environnement)
9. Sécurité
10. Démarrage & exploitation
11. Questions possibles (FAQ / jury)

1. Vue d'ensemble & architecture

SENTINEL-X est un système de surveillance IoT intelligent : une carte ESP32 équipée de capteurs mesure l'environnement, transmet ses mesures en MQTT, et un PC les analyse (IA predictive + vision par ordinateur) pour produire un état de sécurité unique (vert / jaune / rouge), affiche dans une application de bureau et renvoie vers l'écran OLED et les LEDs de la carte.

Chaine de traitement (flux de données)

```

ESP32 (DHT22 + MQ-2 + PIR + OLED + LEDs)
| WiFi 2.4 GHz -- MQTT (topic sentinel/sensors) -->
Broker MQTT Mosquitto (Docker, port 1883 dev / 8883 TLS)
| ingestion (scripts/mqtt_client.py, verif HMAC) -->
Base SQLite data/sentinel.db (table sensor_data)
| |
v v
IA predictive (Brique 6) Vision YOLOv8n (Brique 5)
ensemble + correlation data/vision_status.json
| |
+-----+-----+
v
API Flask (api/server.py, port 3000)
etat consolide vert/jaune/rouge
| |
MQTT sentinel/alerts App bureau (pywebview)
-> OLED + LEDs ESP32 + dashboard web

```

Les briques du projet

- **Brique 3 - Ingestion** : réception MQTT, vérification de signature HMAC, écriture dans SQLite.
- **Brique 5 - Vision** : détection de personnes par YOLOv8n sur le flux webcam, nombre de personnes écrit dans vision_status.json.
- **Brique 6 - IA predictive** : scoring d'anomalies (ensemble de modèles) et fusion multi-capteurs scientifique (correlation.py).
- **Brique 7 - API & stockage** : API REST Flask, stockage des alertes, dashboard sécurisé, assistant IA.
- **App bureau** : fenêtre pywebview (Edge WebView2 sous Windows) qui affiche le cockpit tout-en-un.

2. Materiel ESP32 & capteurs

La carte ESP32 lit trois capteurs, affiche l'etat en local sur un ecran OLED et trois LEDs, et communique en WiFi 2.4 GHz (l'ESP32 ne gere pas le 5 GHz).

Composant	Role	Broche ESP32	Remarque
DHT22	Temperature + humidite	GPIO 4	Capteur numerique
MQ-2	Gaz / fumee	GPIO 34 (ADC1)	Auto-calibre, valeur normalisee (0 = air propre)
PIR HC-SR501	Presence / mouvement	GPIO 27	Entree INPUT_PULLDOWN
OLED SSD1306	Afficheur local 128x64	I2C SDA 21 / SCL 22	Adresses 0x3C ou 0x3D (scan auto)
LED verte	Etat normal	D26	Tout va bien
LED orange	Avertissement / deconnecte	D14	Alerte jaune
LED rouge	Alerte critique	D13	Alerte rouge

Auto-calibration du capteur de gaz

Le MQ-2 a besoin de chauffer. Au demarrage, le firmware ignore la premiere minute (chauffe) puis mesure la valeur de repos (air propre) comme ligne de base. Les mesures envoyees sont NORMALISEES : 0 = air propre, et la valeur monte quand du gaz/fumee est present. C'est plus lisible et comparable entre cartes.

Robustesse

- L'OLED est detectee par scan I2C : si elle est absente, le systeme continue (capteurs, WiFi, MQTT, LEDs) sans bloquer.
- En cas de perte WiFi, l'ESP32 ne redemarre pas en boucle : il continue en local et tente de se reconnecter.

3. MQTT & securite de la communication (TLS, HMAC)

La communication entre l'ESP32 et le PC passe par un broker MQTT Mosquitto (conteneur Docker). Deux topics sont utilises :

- **sentinel/sensors** : l'ESP32 publie ses mesures (temperature, humidite, gaz, presence) en JSON.
- **sentinel/alerts** : le serveur publie l'etat consolide (severite + contexte) que l'ESP32 recoit pour piloter l'OLED et les LEDs.

Chiffrement TLS (MQTTS)

En production, la liaison MQTT est chiffree par TLS sur le port 8883 : cote ESP32 via WiFiClientSecure (certificat du broker embarque), active par l'environnement PlatformIO -secrets. En developpement/demo, un port en clair 1883 (DEV_PLAIN_MQTT) simplifie les tests. Le choix se fait a la compilation, ce qui garde le code retrocompatible.

Signature HMAC-SHA256 (anti-injection)

Chaque mesure peut etre signee par l'ESP32 (HMAC-SHA256 via mbedtls) avec un secret partage. A l'ingestion (scripts/mqtt_client.py), la fonction _signature_valide() controle la signature :

- Pas de secret configure -> signature non verifiee (mode demo, retrocompatible).
- Mode souple (default) : un message sans signature est tolere, mais un message MAL signe est toujours rejete.
- Mode strict (MQTT_HMAC_STRICT=true) : tout message sans signature valide est rejete (production, quand tous les ESP32 signent).

Objectif : empecher qu'un tiers injecte de fausses mesures dans le systeme (integrite et authenticite des donnees).

4. Intelligence artificielle predictive

Deux approches complementaires : un scoring d'anomalies par apprentissage automatique, et une fusion multi-capteurs fondee sur la physique.

Ensemble de modeles (detection d'anomalies)

L'ensemble (Brique 6.3) combine quatre modeles sur les signaux cles (temperature, humidite, gaz, presence) :

- **Isolation Forest** (100 arbres) : aberrances generales, non supervise.
- **Local Outlier Factor (LOF)** : points isolés par rapport au voisinage.
- **PyOD ECOD** : detection non parametrique basee sur la distribution.
- **Gradient Boosting** : classifieur supervise (vue dirigee).

Les scores sont fusionnes ; au-dela du seuil (0.5), la lecture est jugee anormale. Le detecteur classique (IF + LOF) utilise en complement 27 features d'ingenierie (moyennes, ecart, tendances...) calculees sur une fenetre glissante.

Fusion multi-capteurs scientifique (correlation.py)

Plutot que des seuils isolés, le moteur croise temperature, humidite, gaz, PIR et camera pour identifier un SCENARIO reel. Une alerte grave exige plusieurs capteurs coherents, ce qui reduit fortement les faux positifs.

Scenario	Declencheur (fusion)	Couleur
INCENDIE	Gaz eleve + forte chaleur / combustion (temp monte, air s'asseche)	[ROUGE]
FUITE_GAZ	Gaz >= 300 sans montee thermique	[ROUGE]
SURCHAUFFE	Temperature > 32 degres	[ROUGE]
FOULE	Plusieurs personnes au-dela du seuil (camera)	[ROUGE]
GAZ_FAIBLE	Gaz entre 100 et 299	[JAUNE]
PRESENCE	Camera (1 personne) ou mouvement (PIR)	[JAUNE]
CHALEUR	Temperature > 28 degres ou humidex eleve	[JAUNE]
CONDENSATION	Humidite haute + temperature basse	[VERT] info
NORMAL	Toutes les mesures coherentes	[VERT]

Seuils par default (personnalisables par la calibration, section 8)

Capteur	Vert	Jaune (avertissement)	Rouge (critique)
Temperature	<= 28 degres	28 a 32 degres	> 32 degres
Gaz (0 = air propre)	< 100	100 a 299	>= 300
Presence PIR	aucune	mouvement detecte	-
Camera	0 personne	1 personne	2 personnes et +

Assistant Ollama Mistral

Un assistant conversationnel (modele Mistral via Ollama, local) est integre au cockpit. Il recoit en contexte l'etat live du systeme (mesures actuelles, niveau, derniere alerte, presence camera), ce qui lui permet de repondre sur la situation reelle et d'expliquer le projet. Le modele est prechauffe au demarrage pour une premiere reponse rapide.

5. Vision par ordinateur (detection de personnes)

La Brique 5 capture le flux de la webcam (640x480) et detecte les personnes avec YOLOv8n (modele nano, ~3 M de parametres : leger et rapide sur CPU / Apple Silicon). Seules les detections sures (confiance $\geq 50\%$) sont conservees.

- Le nombre de personnes detectees est ecrit en continu dans data/vision_status.json (statut valable quelques secondes).
- Logique par default : 1 personne = jaune, 2 personnes et plus = rouge.
- Ce comportement est PERSONNALISABLE via l'assistant de calibration (ex. boulangerie ou 2 personnes = normal / vert).
- YOLO etant couteux sur CPU, l'inference ne tourne que sur 1 image sur N et le resultat est reutilise entre deux inferences (fluidite).

La camera live est aussi affichee directement dans l'application de bureau.

6. Etat consolide, affichage & alertes

Le coeur du systeme est un ETAT UNIQUE (vert / jaune / rouge), calcule par le serveur a partir des mesures LIVE passees dans le moteur de correlation, combine a la camera en direct. La source la plus grave l'emporte. Comme il repose sur les valeurs actuelles, un etat qui dure (ex. chaleur) reste colore en continu, sans clignotement.

Une seule verite, partout

- **Application / dashboard** : pastille d'etat, camera live, tableau des alertes, chat IA.
- **ESP32** : le serveur publie l'etat sur sentinel/alerts (sur changement + battement regulier). L'OLED affiche une banniere et les LEDs refletent l'etat EN CONTINU (vert / orange / rouge).
- Les textes envoyes a l'OLED sont nettoyes en ASCII (les emojis ne s'affichent pas sur l'ecran).

Gestion des alertes

- Les alertes sont historisees dans SQLite (table alerts) et affichees dans le tableau du dashboard.
- Un agregateur evite les doublons (intervalle minimum entre deux alertes identiques) : pas d'inondation d'alertes.
- L'historique est remis a zero a chaque demarrage (make app) pour repartir propre.

7. API REST

L'API Flask (api/server.py, port 3000, multi-thread) expose les points d'accès suivants :

Methode	Endpoint	Role
GET	/health	Sonde de sante (l'API repond)
GET/POST	/api/v1/alerts	Lire / recevoir une alerte (rediffusee vers l'ESP32)
GET	/api/v1/status	Analyse de connectivite (API, broker, ESP32, webcam, IA, Ollama)
GET	/api/v1/context	Etat live + resume pour l'IA (mesures, niveau, derniere alerte)
GET	/api/v1/alerts-rows	Lignes HTML du tableau d'alertes (rafraichissement)
GET	/api/v1/live-body	Bloc live (etat + mesures) pour l'affichage temps reel
POST	/api/v1/security-fix	Applique les correctifs de securite automatiques
GET/POST	/api/v1/ollama	Question a l'assistant Mistral
GET	/api/v1/ollama-status	Disponibilite d'Ollama / modeles
POST	/api/v1/simulate	Injecte une alerte simulee (demo)
GET	/app /dashboard /security /live /camera	Pages (cockpit, dashboard, securite, camera)
GET/POST	/login /logout	Authentification du dashboard (avec 2FA)

Format d'une alerte (JSON)

```
{
  "source": "ia_predictive",
  "type": "anomaly_detected",
  "confidence": 0.87,
  "timestamp": "2026-10-09T10:15:30Z",
  "details": {
    "severity": "CRITICAL", // INFO | WARNING | CRITICAL
    "context": "Gaz/fumee eleve -> fuite de gaz probable",
    "recommendation": "Ventiler, couper la source de gaz",
    "scenario": "FUIITE_GAZ",
    "current_values": { "temp": 24, "humidity": 48, "gas": 420, "presence": 0 }
  }
}
```

8. Assistant de calibration (adaptation a l'environnement)

Chaque lieu a ses propres normes : une boulangerie est naturellement chaude, un commerce voit plusieurs personnes en permanence (ce qui est normal), un entrepot de nuit ne doit voir personne. L'assistant de calibration apprend ces normes et laisse l'utilisateur fixer ses propres seuils.

Principe en deux temps

- **1. Observation (2 minutes)** : le systeme affiche les valeurs de l'environnement en direct (temperature, humidite, gaz, presence, personnes) et enregistre les minima / maxima observes.
- **2. Questionnaire** : a partir de ces donnees, l'assistant propose des seuils et pose les questions cles. L'utilisateur DECIDE les limites : temperature jaune / rouge, gaz jaune / rouge, nombre de personnes considere comme normal (ex. 2 = OK en boulangerie), et si la presence (PIR) doit etre consideree comme un danger ou non.

Le profil retenu est enregistre (data/profil_env.json) et le moteur de correlation l'utilise a la place des seuils par default. Ainsi, le meme systeme s'adapte a une boulangerie, un bureau ou un entrepot sans changer le code (et sans re-flasher l'ESP32, car les seuils sont cote serveur).

Note : cette fonctionnalite est en cours de mise en place ; le present document decrit le principe retenu.

9. Securite

- **Authentification 2FA** : l'accès au dashboard est protégé par identifiant + mot de passe + code 2FA.
- **TLS (MQTTS)** : communication ESP32 <-> broker chiffré (port 8883).
- **HMAC-SHA256** : intégrité et authenticité des mesures (anti-injection).
- **Secrets non versionnés** : clés et certificats hors du dépôt git.
- **Audit automatisé** (scripts/security_audit.py) : contrôles de sécurité + correctifs applicables en un clic depuis la page /security.

Contrôles de l'audit de sécurité

Contrôle (audit)	Vérifie
secrets_versionnes	Aucune clé privée / secret versionné dans git
dashboard_auth	Le dashboard exige une authentification
xss	Protection contre l'injection HTML (XSS)
mqtt_chiffre	Liaison MQTT chiffré (TLS)
hmac_injection	Signature HMAC active (anti-injection)
api_exposee	L'API n'est pas inutilement exposée
cle_session	Clé de session Flask robuste

10. Demarrage & exploitation

Commandes principales (Windows, depuis le dossier du projet)

```
make app # lance TOUT (broker, API, ingestion, detection,  
# vision) et ouvre le cockpit  
make flash-full # flashe le firmware ESP32 (OLED + LEDs)  
make stop # arrete les services  
make security-fix # applique les correctifs de securite
```

Prerequis

- Un hotspot / WiFi en 2.4 GHz (l'ESP32 ne gere pas le 5 GHz).
- Docker (pour le broker MQTT Mosquitto).
- Ollama + modele Mistral installes (assistant IA) : ollama pull mistral.

Depannage courant

- **Pas de donnees / etat fige** : verifier que le broker tourne (conteneur Docker) ; le relancer avec make app.
- **ESP32 ne se connecte pas** : verifier le WiFi 2.4 GHz et l'adresse IP du PC saisie sur la carte (portail de configuration).
- **Rien n'arrive sur le PC** : ouvrir le port 1883 dans le pare-feu Windows.
- **OLED illisible / absente** : verifier le cablage I2C (SDA 21, SCL 22, 3V3, GND) ; le systeme fonctionne meme sans OLED.

11. Questions possibles (FAQ / jury)

Comment evitez-vous les faux positifs ?

Par la fusion multi-capteurs : une alerte grave exige plusieurs capteurs coherents (ex. gaz + chaleur + air qui s'assèche pour un incendie), pas un seul seuil. L'etat repose sur les valeurs live, et un agregateur evite les alertes en rafale.

Pourquoi un ensemble de modeles plutot qu'un seul ?

Chaque modele a des angles morts. En combinant Isolation Forest, LOF, ECOD et Gradient Boosting, on couvre a la fois les aberrances generales, les points isolés et une vue supervisee : la detection est plus robuste.

Comment securisez-vous la communication ?

Deux niveaux : le chiffrement TLS (MQTTs, port 8883) qui protege la confidentialite, et la signature HMAC-SHA256 qui garantit l'integrite et l'authenticite des mesures (un attaquant ne peut pas injecter de fausses donnees sans le secret).

Que se passe-t-il si l'ESP32 perd le WiFi ?

Il ne redemarre pas en boucle : il continue a mesurer en local (OLED + LEDs) et tente de se reconnecter en arriere-plan. La LED orange signale l'etat deconnecte.

Comment l'IA connait-elle le contexte du projet et la situation ?

L'assistant Mistral recoit a chaque question un resume de l'etat live (mesures actuelles, niveau vert/jaune/rouge, derniere alerte, presence camera) ainsi qu'une description du systeme : il repond donc sur la situation reelle, pas dans le vide.

Comment adapter le systeme a un nouvel environnement ?

Via l'assistant de calibration : 2 minutes d'observation puis un questionnaire ou l'on fixe les seuils (temperature, gaz, nombre de personnes normal, presence dangereuse ou non). Le profil est sauvegarde et applique sans re-flasher la carte.

Pourquoi la valeur de gaz est-elle normalisee ?

Le MQ-2 a une valeur de repos qui varie selon la carte et la chauffe. En calibrant a 0 (air propre) au demarrage, les mesures deviennent comparables et lisibles : 0 = propre, et ca monte avec le gaz.

Comment l'etat de l'OLED et des LEDs reste-t-il synchronise avec l'app ?

Le serveur calcule un etat unique et le publie vers l'ESP32 (topic sentinel/alerts) sur changement et par battement regulier. App, OLED et LEDs lisent donc la meme verite ; les LEDs refletent l'etat en continu.

La presence d'une personne declenche-t-elle toujours une alerte ?

Non : c'est configurable. Par default une presence est un avertissement (jaune), mais dans un lieu ouvert au public on peut declarer la presence comme normale (verte) via la calibration.

Quelle est la difference entre jaune et rouge ?

Jaune = avertissement (situation a surveiller : chaleur, gaz leger, presence). Rouge = critique (danger avere : incendie, fuite de gaz, surchauffe, foule). Chaque niveau a sa LED et sa couleur dans l'app.

Pourquoi YOLOv8n et pas un modele plus gros ?

YOLOv8n (nano) est le plus leger de la famille : il tourne sur CPU sans GPU dedie, ce qui convient a un PC de terrain. La confiance minimale (50 %) evite les fausses detections.

Le systeme fonctionne-t-il si un composant tombe (OLED, camera, Ollama) ?

Oui, chaque brique se degrade proprement : pas d'OLED -> le reste tourne ; pas de camera -> seuls les capteurs comptent ; pas d'Ollama -> l'etat et les alertes fonctionnent, seul le chat IA est indisponible.

Comment distinguez-vous un incendie d'une simple fuite de gaz ?

Par la signature physique : un incendie combine gaz eleve, temperature qui monte ET humidite qui baisse (l'air s'assèche). Une fuite de gaz montre du gaz sans cette montee thermique. Deux scenarios, deux messages.

Où sont stockees les donnees ?

Les mesures capteurs et les alertes sont dans une base SQLite locale (data/sentinel.db). Les modeles IA sont dans models/, le statut camera dans data/vision_status.json.